

ESERCIZI

Martedì 10 Marzo 2026



ESERCIZI

1. Scrivere un programma in Java che
 - Si calcoli la trasposta di una matrice quadrata
 - E si visualizzi tale matrice
2. Implementare in java gli algoritmi per calcolare la sottosequenza massimale di un vettore, allocato dinamicamente.
3. Scrivere un programma in java che, dato un vettore di interi allocato dinamicamente e riempito con valori casuali compresi tra -100 e 100, trovi il numero delle triple di elementi la cui somma è uguale a zero. Qual è la complessità di tempo della vostra soluzione?
 - Esempio: 30, -40, -20, -10, 40, 0, 10, 5

ESERCIZIO: PROBLEMA 3-SUM

- Data una lista di N **interi**, scrivere un algoritmo che trovi quante triple hanno somma pari a 0? Trovare la complessità di tempo della soluzione e misurare il tempo di esecuzione al variare della taglia dell'array in input con valori casuali.
- Esempio: 30, -40, -20, -10, 40, 0, 10, 5
 - Soluzione: 4

	a[i]	a[j]	a[k]	sum
1	30	-40	10	0
2	30	-20	-10	0
3	-40	40	0	0
4	-10	0	10	0

ESERCIZIO: PROBLEMA 3-SUM

- PRIMA SOLUZIONE: **Brute force** $\rightarrow O(n^3)$

```
class ThreeSumBruteForce {  
  
    public static int countThreeSum(int[] arr) {  
        int n = arr.length;  
        int count = 0;  
  
        for (int i = 0; i < n - 2; i++) {  
            for (int j = i + 1; j < n - 1; j++) {  
                for (int k = j + 1; k < n; k++) {  
                    if (arr[i] + arr[j] + arr[k] == 0) {  
                        count++;  
                    }  
                }  
            }  
        }  
  
        return count;  
    }  
}
```

```
// If no triplet with 0 sum found in array  
if (found == false)  
System.out.println("not exist");
```

```
public static void main(String[] args) {  
  
    int[] arr = {30, -40, -20, -10, 40, 0, 10, 5};  
  
    int result = countThreeSum(arr);  
  
    System.out.println("Numero di triple: " + result);  
}
```

Per contare le triple devo aggiungere un contatore che incremento qui

Ex: modificare il codice in modo da contare e restituire tutte le triple

ESERCIZIO: PROBLEMA 3-SUM

Soluzione Empirica Migliorata – $O(n^2)$

Per ogni coppia (i, j) cerchiamo il valore necessario:

```
import java.util.HashSet;
```

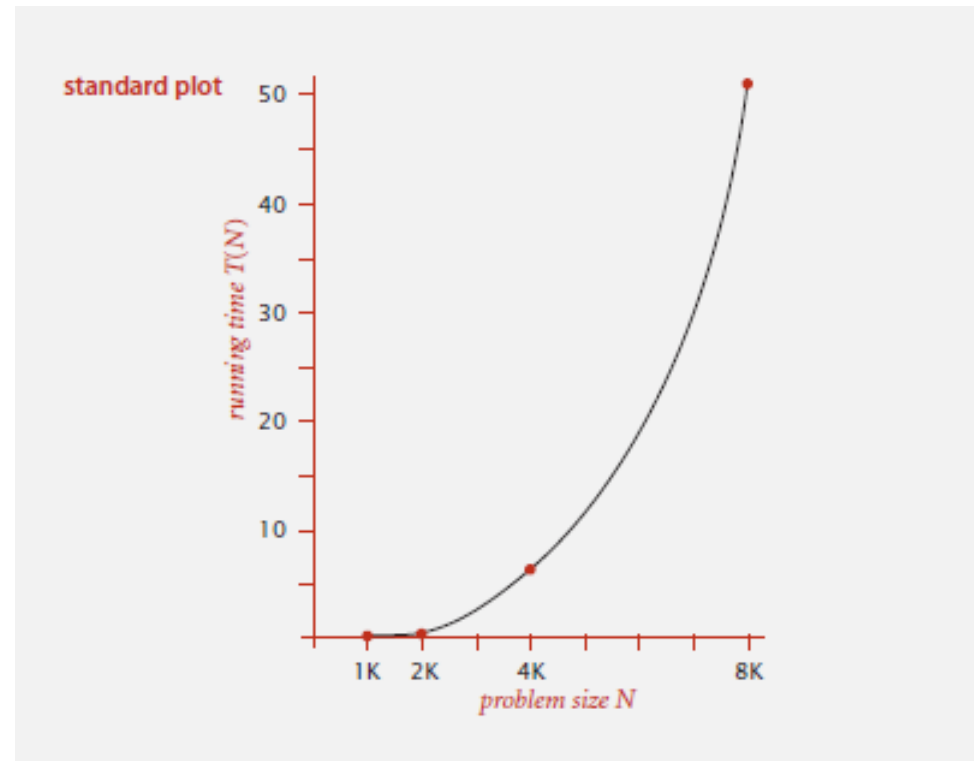
$target = -(arr[i] + arr[j])$

```
public class ThreeSumHash {  
  
    public static int countThreeSum(int[] arr) {  
        int n = arr.length;  
        int count = 0;  
  
        for (int i = 0; i < n - 1; i++) {  
            HashSet<Integer> set = new HashSet<>();  
  
            for (int j = i + 1; j < n; j++) {  
                int target = -(arr[i] + arr[j]);  
  
                if (set.contains(target)) {  
                    count++;  
                }  
  
                set.add(arr[j]);  
            }  
        }  
  
        return count;  
    }  
}
```



ANALISI EMPIRICA

N	time (seconds) †
250	0.0
500	0.0
1,000	0.1
2,000	0.8
4,000	6.4
8,000	51.1
16,000	?



+dipende dalla macchina su cui si esegue il programma

ESERCIZIO

- Trovare una soluzione più efficiente per il problema 3-SUM.

ESERCIZIO: PROBLEMA 3-SUM

Soluzione Ottima – Ordinamento + puntatori $O(n^2)$

- Ordina l'array
- Fissa un elemento
- Usa due puntatori



ESERCIZIO: PROBLEMA 3-SUM

```
import java.util.Arrays;

public class ThreeSumOptimal {

    public static int countThreeSum(int[] arr) {
        Arrays.sort(arr);
        int n = arr.length;
        int count = 0;
        for (int i = 0; i < n - 2; i++) {
            int left = i + 1;
            int right = n - 1;

            while (left < right) {
                int sum = arr[i] + arr[left] + arr[right];

                if (sum == 0) {
                    count++;
                    left++;
                    right--;
                }
                else if (sum < 0) {
                    left++;
                }
                else {
                    right--;
                }
            }
        }
        return count;
    }
}
```



UNA SOLUZIONE ALTERNATIVA PER 3-SUM

- Ordina gli elementi dell'array
- Per ogni coppia di numeri $a[i]$ and $a[j]$, binary search per $-(a[i] + a[j])$

input

30 -40 -20 -10 40 0 10 5

sort

-40 -20 -10 0 5 10 30 40

Qual è la complessità di tempo?

(si può distinguere caso migliore, peggiore e medio, sulla base dell'input)

L'upper bound è $N^2 \log N$

binary search

(-40, -20) 60

(-40, -10) 50

(-40, 0) 40

(-40, 5) 35

(-40, 10) 30

⋮ ⋮

(-20, -10) 30

⋮ ⋮

(-10, 0) 10

⋮ ⋮

(10, 30) -40

(10, 40) -50

(30, 40) -70

only count if
 $a[i] < a[j] < a[k]$
to avoid
double counting

IN PRATICA...

N	time (seconds)
1,000	0.1
2,000	0.8
4,000	6.4
8,000	51.1

Primo algoritmo

N	time (seconds)
1,000	0.14
2,000	0.18
4,000	0.34
8,000	0.96
16,000	3.67
32,000	14.88
64,000	59.16

Versione alternativa

TIME COMPLEXITY: 1-SUM

- C'è uno zero nell'array di N numeri reali?
 - UPPER BOUND: Un algoritmo specifico
 - Ex. Algoritmo Brute-force per 1-SUM: controlla ogni entry dell'array.
 - Running time of the optimal algorithm for 1-SUM is $O(N)$.
 - LOWER BOUND. Provare che nessun algoritmo può fare meglio
 - Ex. Bisogna esaminare tutte le entry
 - Running time della soluzione ottimale per 1-SUM is $\Omega(N)$.
 - OPTIMAL.
 - Lower bound è uguale ad upper bound (a meno di un fattore costante).
 - Ex. Algoritmo Brute-force algorithm per 1-SUM è ottimale: il suo running time è $\Theta(N)$.

TIME COMPLEXITY: 3-SUM

Problema: dato un array di N numeri reali trovare tutte le triple la cui somma fa 0.
E' un problema correlato a problemi di geometria computazionale

- UPPER BOUND: Un algoritmo specifico
 - Running time dell'algoritmo per 3-SUM is $O(N^2 \log N)$.
 - Il migliore algoritmo noto, pubblicato nel 2018, per 3-SUM è $O(N^2(\log(\log N))^{O(1)}/(\log N)^2)$
- LOWER BOUND. Provare che nessun algoritmo può fare meglio
 - Ex. Bisogna esaminare tutte le entry
 - Running time della soluzione ottimale per 3-SUM is $\Omega(N)$.

Congettura

- Congettura: *Il problema 3SUM è $\Omega(N^2)$*
- Problema risolto nel 2014 con la confutazione della congettura.
- Algoritmo ottimale per l'3-SUM?

ANALISI TEORICA

- Costo totale dell'algoritmo: somma dei costi x frequenza di tutte le operazioni.
 - Bisogna determinare il set di operazioni
 - Il costo dipende dalla macchina, dal compilatore
 - La frequenza dipende dall'algoritmo e dall'input

COSTO DELLE OPERAZIONI

- La maggior parte delle operazioni primitive ha un costo costante
 - Dichiarazione di variabile (`int a`) $c1$
 - Assegnazione (`a=b`) $c2$
 - Confronto (`a<b`) $c3$
 - Accesso elemento array (`a[i]`) $c4$
 - Array allocation (`new int[N]`) $c5 \cdot N$
 - Matrix allocation $c6 \cdot N^2$

COSTO DELLE OPERAZIONI

- Quanto valgono queste costanti?

operation	example	nanoseconds †
integer add	<code>a + b</code>	2.1
integer multiply	<code>a * b</code>	2.4
integer divide	<code>a / b</code>	5.4
floating-point add	<code>a + b</code>	4.6
floating-point multiply	<code>a * b</code>	4.2
floating-point divide	<code>a / b</code>	13.5
sine	<code>Math.sin(theta)</code>	91.3
arctangent	<code>Math.atan2(y, x)</code>	129.0
...	...	...

† Running OS X on Macbook Pro 2.2GHz with 2GB RAM

QUANTE OPERAZIONI IN FUNZIONE DI N

- Problema 1-SUM

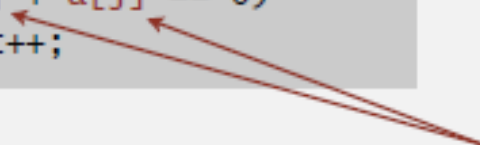
```
int count = 0;  
for (int i = 0; i < N; i++)  
    if (a[i] == 0)  
        count++;
```

 N array accesses

operation	frequency
variable declaration	2
assignment statement	2
less than compare	$N + 1$
equal to compare	N
array access	N
increment	N to $2N$

PROBLEMA 2-SUM

```
int count = 0;
for (int i = 0; i < N; i++)
    for (int j = i+1; j < N; j++)
        if (a[i] + a[j] == 0)
            count++;
```


$$0 + 1 + 2 + \dots + (N - 1) = \frac{1}{2} N (N - 1) \\ = \binom{N}{2}$$

operation	frequency
variable declaration	$N + 2$
assignment statement	$N + 2$
less than compare	$\frac{1}{2} (N + 1) (N + 2)$
equal to compare	$\frac{1}{2} N (N - 1)$
array access	$N (N - 1)$
increment	$\frac{1}{2} N (N - 1) \text{ to } N (N - 1)$

} tedious to count exactly

TILDE NOTATION

- Si trascurano i termini di ordine inferiore

operation	frequency	tilde notation
variable declaration	$N + 2$	$\sim N$
assignment statement	$N + 2$	$\sim N$
less than compare	$\frac{1}{2} (N + 1) (N + 2)$	$\sim \frac{1}{2} N^2$
equal to compare	$\frac{1}{2} N (N - 1)$	$\sim \frac{1}{2} N^2$
array access	$N (N - 1)$	$\sim N^2$
increment	$\frac{1}{2} N (N - 1)$ to $N (N - 1)$	$\sim \frac{1}{2} N^2$ to $\sim N^2$

PROBLEMA 3-SUM

```
int count = 0;  
for (int i = 0; i < N; i++)  
    for (int j = i+1; j < N; j++)  
        for (int k = j+1; k < N; k++)
```

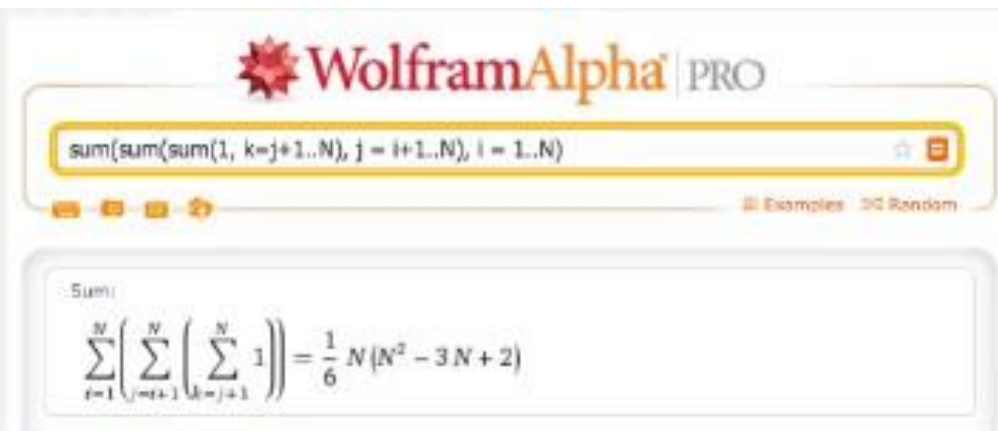
```
            if (a[i] + a[j] + a[k] == 0)  
                count++;
```

← "inner loop"

– $\frac{1}{2} N^3$ array accesses.

$$\binom{N}{3} = \frac{N(N-1)(N-2)}{3!}$$
$$\sim \frac{1}{6} N^3$$

COME STIMARE SOMME DISCRETE?



WolframAlpha PRO

sum(sum(sum(1, k=j+1..N), j = i+1..N), i = 1..N)

Sum:

$$\sum_{i=1}^N \left(\sum_{j=i+1}^N \left(\sum_{k=j+1}^N 1 \right) \right) = \frac{1}{6} N (N^2 - 3N + 2)$$

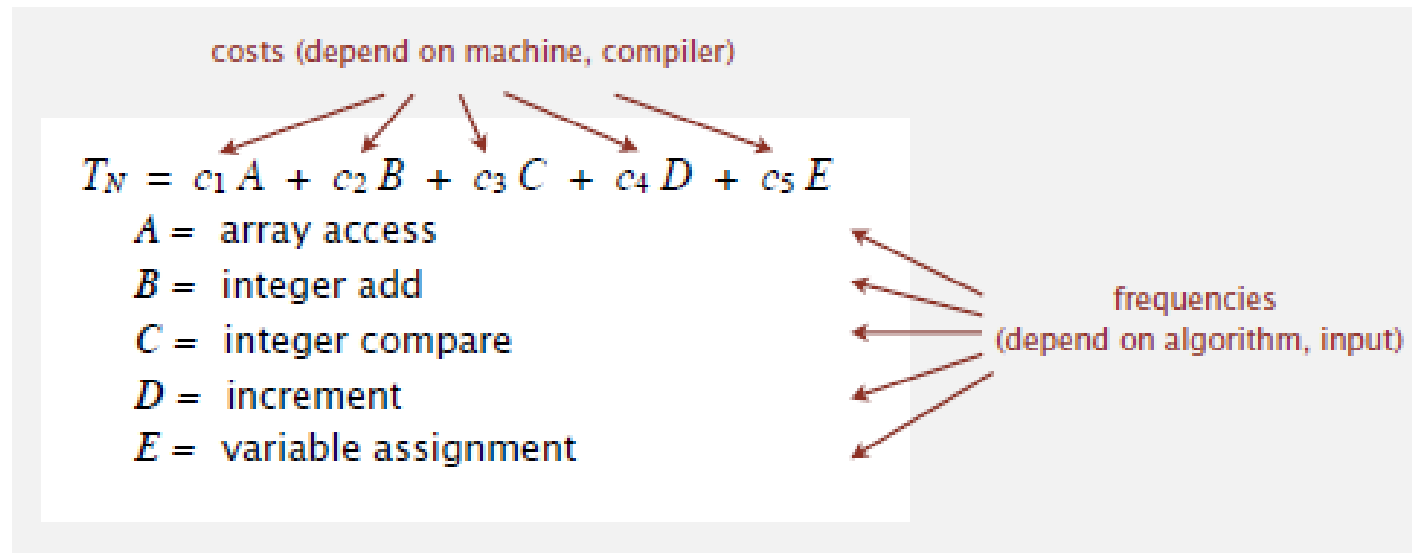
wolframalpha.com

```
[wayne:nobel.princeton.edu] > maple15
  |\\|   Maple 15 (X86 64 LINUX)
._|\\|   |/_._ Copyright (c) Maplesoft, a division of Waterloo Maple Inc. 2011
 \ MAPLE / All rights reserved. Maple is a trademark of
 <____> Waterloo Maple Inc.
  |      Type ? for help.
> factor(sum(sum(sum(1, k=j+1..N), j = i+1..N), i = 1..N));
```

$$\frac{N (N - 1) (N - 2)}{6}$$

CRITERIO DI COSTO UNIFORME

- Il costo di ogni singola operazione non dipende dalla dimensione degli operandi



ESERCIZI

- Scrivere una funzione che inverta una lista concatenata di interi data in input, restituendo un puntatore al nodo finale (della lista originale) che a sua volta punta al penultimo e così via. Quello che prima era il puntatore al primo nodo nella lista originale sarà posto a NULL. Qual è la complessità dell'algoritmo?
- Scrivere una funzione che riordini una lista concatenata in modo tale che i nodi in posizione pari appaiano dopo quelli in posizione dispari, preservando l'ordine relativo fra i nodi pari e fra i nodi dispari. Qual è la complessità dell'algoritmo?
- Utilizzando le strutture dati studiate, scrivere un programma che data una sequenza di parentesi tonde, quadre e graffe sinistre e destre da input e determini se tale sequenza è correttamente bilanciata.
Per esempio, il programma dovrebbe stampare true per `[(){}[000]0]` e false per `[()]`.
- Scrivere un programma che, date le visite in preordine e in ordine simmetrico, costruisca l'albero corrispondente.
- Scrivere un programma che, date le visite in postordine e in ordine simmetrico di un albero, restituisca la visita in preordine dell'albero.
- Scrivere un programma che testi se un albero binario è lineare, ovvero se ogni nodo ha al più un solo figlio
- Scrivere un programma che, dato un albero binario, ne calcoli l'altezza
- Scrivere un programma che, dato un albero binario, ne costruisca lo speculare.
- Scrivere un programma che, dato un albero binario, lo trasformi nel suo speculare.



ESERCIZIO PER CASA

- Chiamiamo **albero di Archimede** un albero binario associato ad una sequenza S di numeri naturali positivi distinti tale che
 1. la visita in ordine simmetrico dell'albero produce gli elementi della sequenza S
 2. Il numero naturale memorizzato in ogni nodo è maggiore del valore memorizzato nel nodo padre
- Scrivere una funzione `ArchimedeTree` che, data una sequenza S di numeri naturali positivi distinti, costruisca in modo efficiente l'albero di Archimede ad essa associato.
- Descrivere e motivare il tempo di calcolo dell'algoritmo utilizzato.

